

✓ Lecture 6: Supervised Fine-Tuning Lab

What Are We Doing Today?

In this lab, we will learn how to *modify the behavior* of a large language model using **Supervised Fine-Tuning (SFT)**.

Large language models (LLMs) are first trained on massive text corpora using next-token prediction. However, pretraining alone does not make them good assistants. To make models follow instructions and respond in desired formats, we apply **fine-tuning**.

Today, we will:

- Understand what a language model really does
- See how supervised learning applies to LLMs
- Fine-tune a small instruct model
- Observe how the model's behavior changes after training

Where Does SFT Fit in the LLM Pipeline?

Modern instruction-following models are typically trained in stages:

1. **Pretraining** – Learn general language patterns
2. **Supervised Fine-Tuning (SFT)** – Learn how to follow instructions
3. **RLHF (Reinforcement Learning from Human Feedback)** – Align behavior with human preferences

In this session, we focus on **Stage 2: SFT**.

What Is Supervised Fine-Tuning?

At its core, SFT is just **supervised learning**:

- Input: A prompt
- Target: The correct response
- Objective: Minimize cross-entropy loss

There is no magic — we simply adjust the model's parameters so that it assigns higher probability to the desired outputs.

Comparison between Pretraining and SFT

Objective

- **Pretraining**
Learn general language modeling ability by predicting the next token over large-scale raw text.
- **SFT**
Adapt the pretrained model to follow instructions, produce structured outputs, or behave in a task-specific way.

Data Structure

- **Pretraining**
Unstructured continuous text.
No explicit separation between input and output.
- **SFT**
Structured input-output pairs.
Clear distinction between prompt and completion.

Dataset Scale

- **Pretraining**
Billions to trillions of tokens.
- **SFT**
Thousands to millions of examples (much smaller scale).

What Is Learned

- **Pretraining**
 - Grammar
 - World knowledge
 - Language patterns
 - General reasoning structure
- **SFT**
 - Instruction following
 - Output formatting
 - Task behavior
 - Domain adaptation

Parameter Updates

- **Pretraining**
Train all model parameters from scratch.
- **SFT**
Start from pretrained weights.
Update parameters slightly (often via LoRA or other parameter-efficient methods).

What Should You Understand After This Tutorial?

By the end, you should be able to explain:

- What a language model predicts
- What “fine-tuning” actually updates
- How loss is computed
- Why model behavior can be changed

Let's begin.

1. What Is a Language Model?

Before we fine-tune a model, we need to understand what it actually does.

A language model predicts the next token given previous tokens:

$$P(\text{next token}|\text{previous tokens})$$

Let's load a model and inspect its behavior.

```
# Install dependencies (run once)
!pip -q install -U "transformers>=4.48.0" "accelerate>=0.34.0" "trl>=0.11.0" "datasets>=2.20.0"
```

```
_____ 10.4/10.4 MB 53.6 MB/s eta 0:00:00
_____ 528.8/528.8 kB 27.1 MB/s eta 0:00:00
_____ 520.7/520.7 kB 27.8 MB/s eta 0:00:00
_____ 47.6/47.6 MB 16.1 MB/s eta 0:00:00
```

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch.nn.functional as F
```

```
print("CUDA available:", torch.cuda.is_available())
```

```
CUDA available: True
```

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

model_name = "Qwen/Qwen3-0.6B"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    dtype=torch.float16,
    device_map="auto",
    trust_remote_code=True,
)
```

```
model.eval()
print("Loaded:", model_name)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret 'HF_TOKEN' does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/t)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100% 726/726 [00:00<00:00, 79.6kB/s]
tokenizer_config.json: 9.73k/? [00:00<00:00, 282kB/s]
vocab.json: 2.78M/? [00:00<00:00, 33.6MB/s]
merges.txt: 1.67M/? [00:00<00:00, 18.4MB/s]
tokenizer.json: 0% 28.3k/11.4M [00:01<00:43, 260kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limit
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a
model.safetensors: 100% 1.50G/1.50G [00:15<00:00, 225MB/s]
Loading weights: 100% 311/311 [00:03<00:00, 124.93it/s, Materializing param=model.norm.weight]
The tied weights mapping and config for this model specifies to tie model.embed_tokens.weight to lm_head.weight,
generation_config.json: 100% 239/239 [00:00<00:00, 8.00kB/s]
Loaded: Qwen/Qwen3-0.6B
```

```
prompt = "Cats are"
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
print(tokenizer.convert_ids_to_tokens(inputs["input_ids"][0]))
```

```
with torch.no_grad():
    outputs = model.generate(**inputs, max_new_tokens=20)
```

```
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

```
['C', 'ats', 'Gare']
```

Cats are known to be very intelligent, and scientists have tried to study them. Which of the following statements

```
with torch.no_grad():
    raw_outputs = model(**inputs)
```

```
print(raw_outputs.logits.shape)
```

```
torch.Size([1, 3, 151936])
```

2. Cross-Entropy Loss

Why Do We Need a Loss?

Earlier, we saw that a language model predicts the next token by producing a probability distribution over the vocabulary.

For example, given the prompt:

"Cats are"

the model might output:

- cute: 0.6
- strong: 0.3
- cake: 0.1

Suppose the correct next token is:

cute

Now we must ask an important question:

How do we measure whether the model's prediction is good or bad?

We need a numerical score that tells us how wrong the model currently is. This score is called the **loss**. In language modeling, the standard choice for this loss is **cross-entropy**.

```
import math
import torch
import torch.nn.functional as F
```

Step 1 — Define a small vocabulary

We define a tiny vocabulary of 9 tokens.

```
vocab = ["Cats", "Dogs", "Fish", "is", "are", "cute", "strong", "cake", "<eos>"]

token_to_id = {token: i for i, token in enumerate(vocab)}
```

Step 2 — Define the target sequence

We want the model to generate:

Cats are cute

At each step, the model must predict:

1. are
2. cute
3. <eos>

```
labels = torch.tensor([
    token_to_id["are"],
    token_to_id["cute"],
    token_to_id["<eos>"]
])
```

Step 3 — Simulated Model Logits

We simulate the model's predictions at three time steps.

Each row corresponds to one time step. Each column corresponds to one token.

Target sequence: Cats are cute

So the correct tokens are:

Step	Correct Token
1	are
2	cute
3	<eos>

Logits (before softmax)

Step	Cats	Dogs	Fish	is	are	cute	strong	cake	<eos>
1	1.0	0.5	0.2	0.1	3.5	0.2	0.1	0.1	0.1
2	0.2	0.1	0.1	0.2	0.3	2.0	1.5	0.1	0.2
3	0.1	0.1	0.1	0.1	0.2	0.1	3.0	0.1	1.0

```
logits_all = torch.tensor([
    [1.0, 0.5, 0.2, 0.1, 3.5, 0.2, 0.1, 0.1, 0.1],
    [0.2, 0.1, 0.1, 0.2, 0.3, 2.0, 1.5, 0.1, 0.2],
    [0.1, 0.1, 0.1, 0.1, 0.2, 0.1, 3.0, 0.1, 1.0]
])
```

Step 4 — Convert Logits to Probabilities

We now apply softmax to each row.

```
import pandas as pd
probs = torch.softmax(logits_all, dim=-1)

for t in range(3):
    df = pd.DataFrame({
        "Token": vocab,
        "Probability": probs[t].numpy()
    }).sort_values(by="Probability", ascending=False)

    print(f"\nStep {t+1} Probability Distribution")
    display(df)
```

Step 5 — Cross-Entropy

Cross-entropy for each step is:

$$-\log p_{\theta}(y_t | y_{<t})$$

The loss used in SFT is the token-level cross-entropy loss, defined as:

$$L_{\text{SFT}}(\theta) = - \sum_{t=1}^T \log p_{\theta}(y_t | y_{<t})$$

```
labels = torch.tensor([4, 5, 8])
losses = torch.nn.functional.cross_entropy(logits_all, labels, reduction="none")

loss_table = pd.DataFrame({
    "Step": [1, 2, 3],
    "Correct Token": ["are", "cute", "<eos>"],
    "Loss": losses.numpy()
})

display(loss_table)

print("Total sequence loss:", losses.sum().item())
```

Step 6 — Fix Step 3

Let's correct Step 3 by increasing the logit of <eos>.

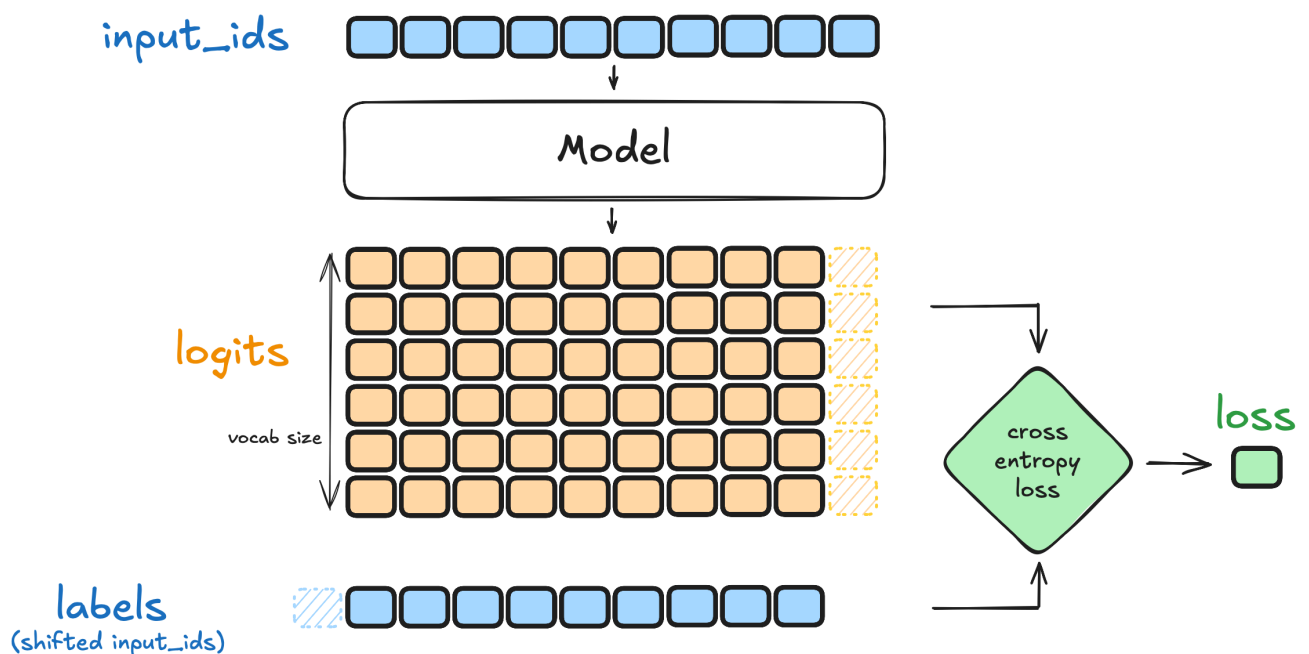
```
logits_fixed = torch.tensor([
    [1.0, 0.5, 0.2, 0.1, 3.5, 0.2, 0.1, 0.1, 0.1],
    [0.2, 0.1, 0.1, 0.2, 0.3, 2.0, 1.5, 0.1, 0.2],
    [0.1, 0.1, 0.1, 0.1, 0.2, 0.1, 0.1, 0.1, 4.0]
])

losses_fixed = torch.nn.functional.cross_entropy(logits_fixed, labels, reduction="none")

comparison = pd.DataFrame({
    "Step": [1, 2, 3],
    "Old Loss": losses.numpy(),
    "New Loss": losses_fixed.numpy()
})

display(comparison)

print("Old total loss:", losses.sum().item())
print("New total loss:", losses_fixed.sum().item())
```



3. From Cross-Entropy to SFT

In the previous section, we introduced cross-entropy loss as a way to measure how good or bad a model's prediction is. Now we ask a deeper question:

What does it mean to *train* a language model using that loss?

The Role of Cross-Entropy

Recall the objective:

$$L(\theta) = - \sum_{t=1}^T \log p_{\theta}(y_t | y_{<t})$$

This loss becomes smaller when:

- The model assigns higher probability to the correct next token
- The model assigns lower probability to incorrect tokens

In other words:

Minimizing cross-entropy increases the probability of desired outputs.

This is the only mechanism behind SFT.

What Changes During SFT?

When we perform SFT:

- Model parameters θ are updated
- The probability of desired outputs increases
- The probability of undesired outputs decreases

Conceptually:

SFT reshapes the model's probability distribution.

It is important to clarify:

- Lower loss does not mean deeper understanding
- Lower loss does not mean improved reasoning algorithms
- Lower loss means better imitation of the training data

SFT teaches the model:

"In this situation, disired outputs tend to look like this."

How Is This Implemented in Practice?

In real systems, libraries such as TRL (https://huggingface.co/docs/trl/en/sft_trainer) provide tools that automate:

- Tokenization
- Label shifting
- Masking padding tokens
- Computing cross-entropy
- Performing backpropagation

For example:

```
from trl import SFTTrainer
from datasets import load_dataset

trainer = SFTTrainer(
    model="Qwen/Qwen3-0.6B",
    train_dataset=load_dataset("trl-lib/Capybara", split="train"),
)

trainer.train()
```

Although the code appears simple, the underlying process is still:

1. Predict next tokens
2. Compute cross-entropy loss
3. Reduce that loss
4. Adjust probabilities

SFT is not a new algorithm.

It is:

Next-token prediction on carefully designed data.

The power of SFT does not come from the training loop.

It comes from the choice of **data**.

4. A Parameter-Efficient Fine-Tuning Method--LoRA

In the previous section, we saw that SFT updates model parameters to reduce cross-entropy loss.

But how many parameters does a modern language model have?

Even a small model like Qwen3-0.6B has hundreds of millions of parameters.

Updating all of them is:

- Memory intensive
- Computationally expensive
- Slow
- Hard to scale

This is where LoRA becomes useful.

The Core Problem

Full fine-tuning means: All parameters θ are updated.

For large models (7B, 13B, 70B), this quickly becomes impractical:

- Too much GPU memory
- Too much storage for checkpoints
- Too slow for iteration

We need a more efficient way to adapt the model.

The Core Idea of LoRA

LoRA stands for: Low-Rank Adaptation.

Instead of updating the full weight matrix W , we keep W ($d \times d$) frozen and learn a small update:

$$W' = W + \Delta W$$

But instead of learning a full ΔW , we approximate it as:

$$\Delta W = AB$$

Where:

- A ($d \times r$) and B ($r \times d$) are small matrices
- r is much smaller than the original dimension d

This dramatically reduces the number of trainable parameters. Instead of rewriting the entire model, LoRA learns a small adjustment that nudges the model's behavior.

LoRA works because large language models are already well pretrained. SFT does not need to reinvent reasoning. It only needs to adjust probabilities toward desired outputs. A low-rank update is often sufficient to achieve this.

LoRA in Practice (TRL Example)

Using LoRA with TRL is simple.

```
from datasets import load_dataset
from trl import SFTTrainer
from peft import LoraConfig

dataset = load_dataset("trl-lib/Capybara", split="train")

trainer = SFTTrainer(
    "Qwen/Qwen3-0.6B",
    train_dataset=dataset,
    peft_config=LoraConfig()
)
```

```
trainer.train()
```

The training objective does not change. We still minimize cross-entropy. The only difference is: We update far fewer parameters.

Comparison between SFT and LoRA

1. SFT changes probabilities by reducing loss. LoRA changes how we update parameters.
2. SFT is about the objective. LoRA is about efficiency.

5. A Concrete Example: Fine-Tuning on CommonsenseQA

So far, we discussed SFT conceptually.

Now we look at a concrete example.

We will fine-tune a model using the CommonsenseQA dataset (https://huggingface.co/datasets/tau/commonsense_qa).

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

model_name = "Qwen/Qwen3-0.6B"

tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32,
    device_map="auto",
    trust_remote_code=True,
)
model.eval()
if hasattr(model, "gradient_checkpointing_disable"):
    model.gradient_checkpointing_disable()

if hasattr(model, "config"):
    model.config.use_cache = False

print("Loaded:", model_name)
print("CUDA:", torch.cuda.is_available())
```

`torch\_dtype` is deprecated! Use `dtype` instead!

Loading weights: 100%

311/311 [00:03<00:00, 106.47it/s, Materializing param=model.norm.weight]

The tied weights mapping and config for this model specifies to tie model.embed_tokens.weight to lm_head.weight,

Loaded: Qwen/Qwen3-0.6B

CUDA: True

What Is CommonsenseQA?

CommonsenseQA is a multiple-choice question answering dataset.

Each example contains:

- A question
- Five answer choices
- The correct answer key

For example:

Question: Where would you find a pillow?

Choices: A) refrigerator

B) bedroom

C) garage

D) forest

E) sidewalk

Answer: B

```
from datasets import load_dataset

dataset = load_dataset("tau/commonsense_qa")

train_data = dataset["train"]
```



```

print("Total training examples:", len(train_data))
print()

for i in range(3):
    example = train_data[i]

    print(f"Example {i+1}")
    print("Question:", example["question"])

    print("Choices:")
    for choice in example["choices"]["text"]:
        label = example["choices"]["label"][example["choices"]["text"].index(choice)]
        print(f"{label}) {choice}")

    print("Correct Answer:", example["answerKey"])
    print("-" * 50)

```

README.md: 7.39k/? [00:00<00:00, 256kB/s]

data/train-00000-of-00001.parquet: 100% 1.25M/1.25M [00:00<00:00, 2.33MB/s]

data/validation-00000-of-00001.parquet: 100% 160k/160k [00:00<00:00, 370kB/s]

data/test-00000-of-00001.parquet: 100% 151k/151k [00:00<00:00, 434kB/s]

Generating train split: 100% 9741/9741 [00:00<00:00, 6428.93 examples/s]

Generating validation split: 100% 1221/1221 [00:00<00:00, 61579.33 examples/s]

Generating test split: 100% 1140/1140 [00:00<00:00, 46632.47 examples/s]

Total training examples: 9741

Example 1

Question: The sanctions against the school were a punishing blow, and they seemed to what the efforts the school

Choices:

- A) ignore
- B) enforce
- C) authoritarian
- D) yell at
- E) avoid

Correct Answer: A

Example 2

Question: Sammy wanted to go to where the people were. Where might he go?

Choices:

- A) race track
- B) populated areas
- C) the desert
- D) apartment
- E) roadblock

Correct Answer: B

Example 3

Question: To locate a choker not located in a jewelry box or boutique where would you go?

Choices:

- A) jewelry store
- B) neck
- C) jewelry box
- D) jewelry box
- E) boutique

Correct Answer: A

Defining Our Training Objective

Instead of letting the model answer freely, we define a strict format requirement:

The model must answer in the following format:

```
{Answer: X}
```

For example:

```
{Answer: B}
```

However, this is only **one possible use case of SFT**.

Supervised Fine-Tuning can be used for many different purposes, including:

Behavior Shaping (Style / Persona Constraints)

Example targets:

- Always answer in a concise, polite tone (e.g., limit responses to 3 sentences).
- Always provide answers in the format `{Answer: X}` for multiple-choice questions.

Instruction Following (Command Compliance)

Example targets:

- Follow strict output rules such as:
"Output must be valid JSON with fields `title` and `steps`."
- Refuse to output extra text beyond the requested format (no explanations).

Structured Output (Format Control)

Example targets:

- Extract key fields from text into a schema (e.g., `name`, `date`, `location`).
- Convert informal text into a standardized template (e.g., meeting notes with fixed headings).

Domain Adaptation (Domain-Specific Examples)

Example targets:

- Answer questions using medical, finance, or legal terminology.
- Summarize technical passages using domain-specific conventions.

Task Specialization (Classification / QA / Rewriting)

Example targets:

- Classification with a fixed label set.
- Short QA tasks where answers must cite one of the provided options (A/B/C/D/E).

▼ Reformulating the Dataset

The original dataset looks like this:

```
{
  "question": "...",
  "choices": {...},
  "answerKey": "B"
}
```

We transform it into prompt-completion format:

Prompt:

```
Select the correct answer and respond only in the format:
{Answer: X}
```

Q: Where would you find a pillow?

```
A) refrigerator
B) bedroom
C) garage
D) forest
E) sidewalk
```

Answer:

Completion:

```
{Answer: B}
```

This makes the training objective explicit: We are not teaching reasoning explicitly. We are increasing the probability of producing the correct structured answer.

```
def build_mcq_prompt(question, labels, texts):
    # labels: ["A","B","C","D","E"], texts: choice strings
    choices_block = "\n".join([f"{l}" {t}" for l, t in zip(labels, texts)])
    prompt = (
        "Select the correct answer and respond only in the format:\n"
        "{Answer: X}\n\n"
```

```

        f"Q: {question}\n\n"
        f"{choices_block}\n\n"
        "Answer:\n"
    )
    return prompt

```

Dataset formats accepted by TRL

TRL (`SFTTrainer`) expects one of the following dataset formats:

```

# Standard language modeling
{"text": "The sky is blue."}

# Conversational language modeling
{"messages": [{"role": "user", "content": "What color is the sky?"},
               {"role": "assistant", "content": "It is blue."}]}

# Standard prompt-completion
{"prompt": "The sky is",
 "completion": " blue."}

# Conversational prompt-completion
{"prompt": [{"role": "user", "content": "What color is the sky?"}],
 "completion": [{"role": "assistant", "content": "It is blue."}]}

```

In our tutorial, we will use standard prompt-completion:

- prompt contains the question + options + "Answer:"
- completion is the strict target format, e.g. {Answer: B}

```

train_raw = dataset["train"]
val_raw   = dataset["validation"]

def to_pc(example):
    prompt = build_mcq_prompt(
        example["question"],
        example["choices"]['label'],
        example["choices"]['text'],
    )
    completion = f"{{Answer: {example['answerKey']}}}{tokenizer.eos_token}"
    return {"prompt": prompt, "completion": completion}

N_train = 800
N_val   = 200

train_subset = train_raw.select(range(N_train))
val_subset   = val_raw.select(range(N_val))

train_pc = train_subset.map(to_pc, remove_columns=train_subset.column_names)
val_pc   = val_subset.map(to_pc, remove_columns=val_subset.column_names)

print("Train example:", train_pc[0]["completion"])
print("Validation example:", val_pc[0]["completion"])

```

```

Map: 100%                               800/800 [00:00<00:00, 8408.39 examples/s]
Map: 100%                               200/200 [00:00<00:00, 3308.27 examples/s]
Train example: {Answer: A}<|im_end|>
Validation example: {Answer: A}<|im_end|>

```

```

ex = val_pc[0]["prompt"]

inputs = tokenizer(ex, return_tensors="pt").to(model.device)
with torch.no_grad():
    out = model.generate(*inputs, do_sample=False, max_new_tokens=20)
generated_tokens = out[0][inputs["input_ids"].shape[1]:]

print("=== Prompt ===")
print(ex)
print("\n=== Model output (before SFT) ===")
print(tokenizer.decode(generated_tokens, skip_special_tokens=True))

```

The following generation flags are not valid and may be ignored: ['temperature', 'top_p', 'top_k']. Set `TRANSFORMERS` to `True` to enable them.
 Select the correct answer and respond only in the format:

```
{Answer: X}
```

Q: A revolving door is convenient for two direction travel, but it also serves as a security measure at a what?

- A) bank
- B) library
- C) department store
- D) mall
- E) new york

Answer:

```
=== Model output (before SFT) ===
{Answer: D}
```

Okay, let's see. The question is about a revolving door and its

```
import torch
from trl import SFTTrainer, SFTConfig

USE_LORA = True
peft_config = None

if USE_LORA:
    from peft import LoraConfig
    peft_config = LoraConfig(
        r=8,
        lora_alpha=16,
        lora_dropout=0.05,
        bias="none",
        task_type="CAUSAL_LM",
    )

assert tokenizer.eos_token == "<|im_end|>"
assert tokenizer.pad_token == "<|endoftext|>"

config = SFTConfig(
    output_dir="csqa_sft_out",
    per_device_train_batch_size=1,
    gradient_accumulation_steps=8,
    learning_rate=2e-5,
    max_steps=150,
    logging_steps=10,
    save_steps=150,
    fp16=torch.cuda.is_available(),
    report_to="none",
    eos_token=tokenizer.eos_token,
    eval_strategy="steps",
    eval_steps=10,
)

trainer = SFTTrainer(
    model=model,
    processing_class=tokenizer,
    train_dataset=train_pc,
    eval_dataset=val_pc,
    args=config,
    peft_config=peft_config,
)

train_output = trainer.train()

model_after = trainer.model
model_after.eval()

print(train_output)
```

Adding EOS to train dataset: 100% 800/800 [00:00<00:00, 11641.95 examples/s]

Tokenizing train dataset: 100% 800/800 [00:00<00:00, 1634.66 examples/s]

Truncating train dataset: 100% 800/800 [00:00<00:00, 41733.85 examples/s]

Adding EOS to eval dataset: 100% 200/200 [00:00<00:00, 4318.90 examples/s]

Tokenizing eval dataset: 100% 200/200 [00:00<00:00, 1535.32 examples/s]

Truncating eval dataset: 100% 200/200 [00:00<00:00, 11616.64 examples/s]

The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model config [150/150 08:06, Epoch 1/2]

Step	Training Loss	Validation Loss
10	3.338922	2.729831
20	2.496816	2.122702
30	1.929552	1.752400
40	1.668062	1.483800
50	1.396699	1.251050
60	1.173853	1.009223
70	0.881198	0.723207
80	0.594203	0.436302

```
ex = val_pc[0]["prompt"]
inputs = tokenizer(ex, return_tensors="pt").to(model_after.device)

with torch.no_grad():
    out = model_after.generate(
        **inputs,
        do_sample=False,
        max_new_tokens=20,
    )

generated_tokens = out[0][inputs["input_ids"].shape[1]:]
generated_text = tokenizer.decode(generated_tokens, skip_special_tokens=True)

print("=== Test Prompt ===")
print(ex)

print("\n=== Model Output (after SFT) ===")
print(generated_text)
```

```
=== Test Prompt ===
Select the correct answer and respond only in the format:
{Answer: X}
```

Q: A revolving door is convenient for two direction travel, but it also serves as a security measure at a what?

- A) bank
- B) library
- C) department store
- D) mall
- E) new york

Answer:

```
=== Model Output (after SFT) ===
{Answer: C}
```

Technical Quiz- From Jingwei Hu

Data: 1000 Tang Poems

Data Source: GitHub(<https://github.com/chinese-poetry>)

Description:

The dataset used in this project comes from the public dataset of the Chinese-poetry project on GitHub. In this project, the top 1,000 Tang poems were selected as the dataset of the project. The original dataset is stored in JSON format. Each JSON file contains 1,000 poems. Each poem contains the title, author, and paragraph of the poem, and is stored in traditional Chinese. Due to the requirements of this project, I first converted traditional Chinese characters to simplified Chinese, and then called glm-4 to construct a QA pair based on the content of the ancient poems.

```
In [11]: import json
import time
import random
import logging
from pathlib import Path
from concurrent.futures import ThreadPoolExecutor, as_completed

from opencv import OpenCC
from sklearn.model_selection import train_test_split
from zhipuai import ZhipuAI

API_KEY = "api_key"
```

```

INPUT_FILE = Path("poet.song.0.json")
OUTPUT_FILE = Path("qa_dataset.json")
MAX_RETRIES = 3
MAX_WORKERS = 5

cc = OpenCC("t2s")
client = ZhipuAI(api_key=API_KEY)

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

```

Loading Poem

```

In [13]: def load_poems(file_path: Path) -> list[str]:
        poems = []
        with file_path.open("r", encoding="utf-8") as f:
            data = json.load(f)
            for item in data:
                text = "\n".join(item["paragraphs"])
                text += f"\n— {item['author']} 《{item['title']}》 "
                poems.append(cc.convert(text))
        return poems
poems = load_poems(INPUT_FILE)
poems[:10]

```

```

Out[13]: ['欲出未出光辣达，千山万山如火发。\\n须臾走向天上来，逐却残星赶却月。\\n— 宋太祖《日诗》',
'未离海底千山黑，才到天中万国明。\\n— 宋太祖《句》',
'满目江山四望幽，白云高卷嶂烟收。\\n日回禽影穿疏木，风递猿声入小楼。\\n远岫似屏横碧落，断帆如叶截中流。\\n— 幸夤逊《登戎州江楼闲望》',
'片片飞来静又闲，楼头江上复山前。\\n飘零尽日不归去，帖破清光万里天。\\n— 幸夤逊《雪》',
'因登巨石知来处，勃勃元生绿藓痕。\\n静即等闲藏草木，动时顷刻徧乾坤。\\n横天未必朋元恶，捧日还曾瑞至尊。\\n不独朝朝在巫峡，楚王何事漫劳魂。\\n— 幸夤逊《云》',
'若教作镇居中国，争得泥金在泰山。\\n— 幸夤逊《句 其一》',
'才闻暖律先偷眼，既待和风始展眉。\\n— 幸夤逊《句 其二》',
'嚼处春冰敲齿冷，咽时雪液沃心寒。\\n— 幸夤逊《句 其三》',
'蒙君知重惠琼实，薄起金刀钉玉深。\\n— 幸夤逊《句 其四》',
'深妆玉瓦平无垅，乱拂芦花细有声。\\n— 幸夤逊《句 其五》']

```

Part 2: Data Preprocessing

Description:

Call glm-4 to build QA pairs based on the content of ancient poems and divide the training set and test set into 80:20 ratios

ZhipuAI's glm-4 is invoked to construct QA pairs based on ancient poems

```

In [14]: def process_poem(index: int, poem_text: str) -> dict:
        prompt_template = """
        你是一个古诗问答推荐系统的数据生成助手。

```

你将收到一首古诗，请你：

- 根据诗歌内容构造一个自然、真实、略带口语化的问题，使得这首诗正好是回答；
 - 将这首诗作为回答写出，包含完整诗句、作者、标题；
 - 以标准 JSON 格式输出，结构如下所示。必须严格符合 JSON 规范，不能包含任何注释；
 - 输出的某value是长字段时，必须返回在同一行，不允许使用换行符将一句内容分两行输出
- 输出格式如下（务必输出为合法 JSON 对象）：

```
{ "question": "你生成的问题", "answer": "这首诗（原文 + — 作者《标题》）"
```

生成问题时可以考虑以下角度：

- 题材（如边塞、田园、山水）
- 情绪（如清幽、苍凉、孤寂）
- 场景（如登高、送别、夜晚、楼阁、江边）
- 意象（如白云、远山、猿啼、孤帆）
- 风格（对仗、意境悠远、写景写情）
- 写作意图（抒发感情、写景寄意）

示例1： 古诗： 春潮带雨晚来急，野渡无人舟自横。 — 韦应物《滁州西涧》

输出： { "question": "有没有那种黄昏下雨、带点孤独意境的古诗？", "answer": "春潮带雨晚来急，野渡无人舟自横。 — 韦应物《滁州西涧》"

示例2： 古诗： 大漠孤烟直，长河落日圆。 — 王维《使至塞上》

输出： { "question": "我想找一首描写边塞风光、画面感强烈的诗，有推荐吗？", "answer": "大漠孤烟直，长河落日圆。 — 王维《使至塞上》"

现在请处理以下古诗：

```
---"``" + poem_text + "``"---
```

注意：输出必须是一个合法 JSON 对象，不能包含 markdown、注释或额外文字。只返回 JSON 对象

```
for attempt in range(1, MAX_RETRIES + 1):
    try:
        response = client.chat.completions.create(
            model="glm-4-plus",
            messages=[
                {"role": "system", "content": "你是一个古诗问答推荐系统的数据"},
                {"role": "user", "content": prompt_template}
            ]
        )
        content = response.choices[0].message.content.replace("`json", "")
        try:
            result = json.loads(content)
            logging.info(f"[✓] 成功处理第 {index + 1} 首")
            return result
        except json.JSONDecodeError:
            logging.warning(f"[!] 第 {index + 1} 首 JSON 解析失败，将原始字符返回")
            return {"error": True, "index": index, "raw": content}

    except Exception as e:
        logging.error(f"[X] 第 {index + 1} 首请求异常：{e}")
        logging.info(f"[!] 重试 {attempt}/{MAX_RETRIES} 次...")
        time.sleep(random.uniform(1, 2))

logging.error(f"[!] 第 {index + 1} 首重试失败，跳过此诗。")
return {"error": True, "index": index, "raw": "[重试失败，无返回]"}

with open("qa_template.json", "r", encoding="utf-8") as f:
    qa_data = json.load(f)
qa_data[:10]
```



```
Out[14]: [{ 'question': '有没有哪句诗能形容太阳初升时的壮观景象？ ', 'answer': '未离海底千山黑，才到天中万国明。 — 宋太祖《句》' },
  { 'question': '有没有描写日出时壮观景象的古诗？ ',
    'answer': '欲出未出光辣达，千山万山如火发。须臾走向天上来，逐却残星赶却月。 — 宋太祖《日诗》' },
  { 'question': '有没有一首诗能描绘雪花飘落的静谧和广袤天地的景象？ ',
    'answer': '片片飞来静又闲，楼头江上复山前。飘零尽日不归去，帖破清光万里天。 — 幸夤逊《雪》' },
  { 'question': '有没有描写云的动态变化和神秘感的古诗推荐？ ',
    'answer': '因登巨石知来处，勃勃元生绿藓痕。静即等闲藏草木，动时顷刻徧乾坤。横天未必朋元恶，捧日还曾瑞至尊。不独朝朝在巫峡，楚王何事谩劳魂。 — 幸夤逊《云》' },
  { 'question': '有没有一首诗能描绘站在江楼上远眺的宁静美景，让人感觉心旷神怡？ ',
    'answer': '满目江山四望幽，白云高卷嶂烟收。日回禽影穿疏木，风递猿声入小楼。远岫似屏横碧落，断帆如叶截中流。 — 幸夤逊《登戎州江楼闲望》' },
  { 'question': '有没有描写春天到来、万物复苏的古诗？ ', 'answer': '才闻暖律先偷眼，既待和风始展眉。 — 幸夤逊《句 其二》' },
  { 'question': '有没有哪首古诗表达了希望有人能镇守国家、让泰山更加辉煌的愿望？ ',
    'answer': '若教作镇居中国，争得泥金在泰山。 — 幸夤逊《句 其一》' },
  { 'question': '有没有描写冬天吃冰、感觉特别冷的古诗？ ',
    'answer': '嚼处春冰敲齿冷，咽时雪液沃心寒。 — 幸夤逊《句 其三》' },
  { 'question': '有没有一首诗是表达对别人赠送珍贵礼物的感激之情？ ',
    'answer': '蒙君知重惠琼实，薄起金刀钉玉深。 — 幸夤逊《句 其四》' },
  { 'question': '有没有描写秋日芦花轻拂、意境清幽的古诗？ ',
    'answer': '深妆玉瓦平无垅，乱拂芦花细有声。 — 幸夤逊《句 其五》' }]
```

Divide the dataset

```
In [15]: conversions = []
for index in range(len(qa_data)):
    each = qa_data[index]
    conversion = {
        "id": index,
        "conversations": [
            {
                "from": "user",
                "value": each["question"],
            },
            {
                "from": "assistant",
                "value": each["answer"],
            }
        ]
    }
    conversions.append(conversion)
logging.info("样例展示")
logging.info(conversions[0])

train, eval = train_test_split(conversions, test_size=0.2, random_state=42)
with open("train.json", "w", encoding="utf-8") as f:
    json.dump(train, f, ensure_ascii=False, indent=4)
with open("eval.json", "w", encoding="utf-8") as f:
    json.dump(eval, f, ensure_ascii=False, indent=4)
logging.info(f"训练集: {len(train)} 样本")
logging.info(f"验证集: {len(eval)} 样本")
```

```
2025-04-21 11:29:21,869 - [INFO] - 样例展示
2025-04-21 11:29:21,873 - [INFO] - {'id': 0, 'conversations': [{'from': 'user',
'value': '有没有哪句诗能形容太阳初升时的壮观景象? '}, {'from': 'assistant', 'value': '未离海底千山黑，才到天中万国明。 — 宋太祖《句》'}]}}
2025-04-21 11:29:21,998 - [INFO] - 训练集：800 样本
2025-04-21 11:29:21,999 - [INFO] - 验证集：200 样本
```

Part 3: Fine-Tuning Setup (Model & Training Configuration)

Description:

Fine-tuning using the Qwen-1.8B-Chat model

```
In [16]: import json
import logging
import torch
from torch.utils.data import Dataset
from transformers import Trainer, GPTQConfig, deepspeed, TrainingArguments, AutoTrainerCallback
from transformers.trainer_pt_utils import LabelSmoother
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
import matplotlib.pyplot as plt

model_name = "./Qwen-1.8B-Chat"
train_data_path = "train.json"
eval_data_path = "eval.json"
output_dir = "./lora-qwen-output-1051"
max_seq_len = 1024
```

Load the word segmenter and model, and configure related parameters

```
In [17]: tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)
tokenizer.pad_token_id = tokenizer.eod_id

model = AutoModelForCausalLM.from_pretrained(model_name, trust_remote_code=True)
model.config.use_cache = False
lora_config = LoraConfig(
    r=64,
    lora_alpha=16,
    lora_dropout=0.05,
    bias="none",
    target_modules=["c_attn", "c_proj", "w1", "w2"],
    task_type="CAUSAL_LM"
)
training_args = TrainingArguments(
    output_dir=output_dir,
    per_device_train_batch_size=2,
    num_train_epochs=5,
    logging_steps=1,
    save_steps=100,
    save_total_limit=10,
    learning_rate=3e-4,
    bf16=True,
    remove_unused_columns=False,
```

```

        report_to="none",
    )
    model = get_peft_model(model, lora_config)
    model.print_trainable_parameters()

```

```

2025-04-21 11:29:55,904 - [WARNING] - The model is automatically converting to bf
16 for faster inference. If you want to disable the automatic precision, please m
anually add bf16/fp16/fp32=True to "AutoModelForCausalLM.from_pretrained".
2025-04-21 11:29:55,904 - [WARNING] - Try importing flash-attention for faster in
ference...
2025-04-21 11:29:55,935 - [WARNING] - Warning: import flash_attn rms_norm fail, p
lease install FlashAttention layer_norm to get higher efficiency https://github.c
om/Dao-AILab/flash-attention/tree/main/csrc/layer_norm
Loading checkpoint shards: 100%|██████████| 2/2 [00:45<00:00, 22.66s/it]
trainable params: 53,673,984 || all params: 1,890,502,656 || trainable%: 2.839138
24874309

```

Data preprocessing and dataset loading

```

In [18]: def preprocess(sources, tokenizer, max_len):
    roles = {"user": "<|im_start|>user", "assistant": "<|im_start|>assistant"}
    IGNORE_TOKEN_ID = LabelSmoother.ignore_index
    im_start = tokenizer.im_start_id
    im_end = tokenizer.im_end_id
    nl_tokens = tokenizer('\n').input_ids

    input_ids, targets = [], []
    for source in sources:
        if roles[source[0]["from"]] != roles["user"]:
            source = source[1:]
        input_id, target = [], []

        for sentence in source:
            role = roles[sentence["from"]]
            content = tokenizer(role).input_ids + nl_tokens + tokenizer(sentence
            input_id += content
            if role == roles["user"]:
                target += [IGNORE_TOKEN_ID] * len(content)
            else:
                target += [IGNORE_TOKEN_ID] * len(tokenizer(role).input_ids + nl
                target += tokenizer(sentence["value"]).input_ids + [im_end] + nl
            input_id = input_id[:max_len]
            target = target[:max_len]
            input_ids.append(input_id + [tokenizer.pad_token_id] * (max_len - len(in
            targets.append(target + [IGNORE_TOKEN_ID] * (max_len - len(target)))

    return dict(
        input_ids=torch.tensor(input_ids),
        labels=torch.tensor(targets),
        attention_mask=(torch.tensor(input_ids) != tokenizer.pad_token_id),
    )

class SimpleDataset(Dataset):
    def __init__(self, data, tokenizer, max_len):
        sources = [item["conversations"] for item in data]
        encodings = preprocess(sources, tokenizer, max_len)
        self.input_ids = encodings["input_ids"]
        self.labels = encodings["labels"]
        self.attention_mask = encodings["attention_mask"]

```

```

def __len__(self):
    return len(self.input_ids)

def __getitem__(self, idx):
    return {
        "input_ids": self.input_ids[idx],
        "labels": self.labels[idx],
        "attention_mask": self.attention_mask[idx],
    }

with open(train_data_path, "r") as f:
    raw_data = json.load(f)
train_dataset = SimpleDataset(raw_data, tokenizer, max_seq_len)
with open(eval_data_path, "r") as f:
    raw_data = json.load(f)
eval_dataset = SimpleDataset(raw_data, tokenizer, max_seq_len)

```

Part 4: Model Training and Loss Monitoring

Start training

```

In [19]: class LossHistoryCallback(TrainerCallback):
    def __init__(self):
        self.train_losses = []
        self.eval_losses = []
        self.steps = []

    def on_log(self, args, state, control, logs=None, **kwargs):
        if logs is not None:
            if "loss" in logs:
                self.train_losses.append(logs["loss"])
                self.steps.append(state.global_step)
            if "eval_loss" in logs:
                self.eval_losses.append(logs["eval_loss"])

loss_callback = LossHistoryCallback()
trainer = Trainer(
    model=model,
    tokenizer=tokenizer,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    callbacks=[loss_callback]
)
trainer.train()
trainer.evaluate()
model.save_pretrained(output_dir)

```

Step	Training Loss
1	4.729400
2	4.220100
3	3.768200
4	4.110700
5	3.338700
6	3.975600
7	3.671800
8	2.813000
9	4.148800
10	4.347500
11	3.724100
12	3.963600
13	3.757800
14	4.248100
15	3.799700
16	3.699400
17	5.124800
18	3.677900
19	3.732500
20	4.655400
21	4.474800
22	3.723200
23	4.008100
24	3.805000
25	3.582900
26	3.969700
27	4.544400
28	3.724600
29	3.801000
30	3.480100
31	4.010000
32	3.900100
33	4.162300

Step	Training Loss
1972	0.667000
1973	0.922700
1974	0.478300
1975	0.776300
1976	0.266100
1977	0.518700
1978	0.407500
1979	0.607800
1980	0.923300
1981	0.232400
1982	0.491900
1983	0.568500
1984	0.336700
1985	0.480100
1986	0.516800
1987	1.738500
1988	0.486700
1989	0.374100
1990	0.539500
1991	0.342200
1992	0.820200
1993	0.366000
1994	0.978900
1995	0.252700
1996	0.328900
1997	0.603500
1998	0.700600
1999	0.574600
2000	0.488300

 [25/25 01:17]

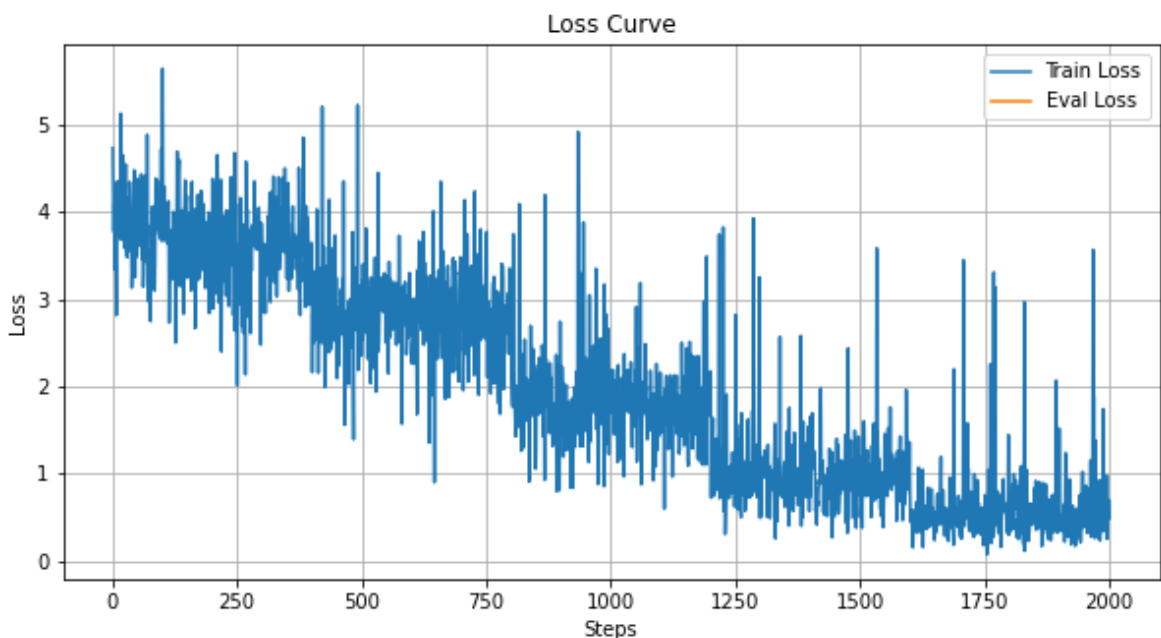
```
In [22]: print("训练LOSS: ", loss_callback.train_losses)
         print("评估LOSS: ", loss_callback.eval_losses)
```

54, 0.5685, 0.4649, 0.4302, 0.5955, 0.423, 0.8287, 0.4551, 0.5284, 0.9176, 0.3546, 0.6866, 0.9019, 0.5071, 0.372, 0.6565, 0.9013, 0.9025, 0.7122, 0.4438, 0.6485, 0.1686, 0.3762, 0.3499, 0.8905, 0.4236, 0.5797, 0.3742, 0.7061, 0.8775, 0.5646, 0.5698, 0.3136, 0.5088, 0.7079, 0.2456, 0.698, 0.38, 0.3128, 0.2437, 0.2368, 0.4515, 0.5677, 0.7576, 0.3317, 0.4297, 0.3938, 0.493, 0.6841, 0.5639, 2.0671, 0.3263, 0.708, 0.4354, 0.5386, 0.3314, 1.5173, 0.5932, 0.504, 0.3763, 0.5005, 0.5001, 0.3406, 0.5309, 0.409, 0.4088, 0.6484, 0.2185, 0.8006, 1.2353, 0.6207, 0.4868, 0.3479, 0.274, 0.5103, 0.7192, 0.4828, 0.9357, 0.4946, 0.5654, 0.1883, 0.2546, 0.4787, 0.6323, 0.5866, 0.4711, 0.5866, 0.3858, 0.1711, 0.1798, 0.4371, 0.597, 0.2584, 0.3642, 0.3411, 0.4647, 0.5711, 0.6712, 0.7573, 0.2132, 0.4779, 0.7118, 0.7352, 1.0209, 0.381, 0.5501, 0.5422, 0.4672, 0.4534, 0.3986, 0.8637, 0.4896, 0.4441, 0.3182, 0.6407, 0.8325, 0.6158, 0.6846, 0.3961, 1.154, 0.2788, 0.5306, 0.5894, 1.1663, 3.5631, 0.3758, 1.8836, 0.2623, 1.3809, 0.667, 0.9227, 0.4783, 0.7763, 0.2661, 0.5187, 0.4075, 0.6078, 0.9233, 0.2324, 0.4919, 0.5685, 0.3367, 0.4801, 0.5168, 1.7385, 0.4867, 0.3741, 0.5395, 0.3422, 0.8202, 0.366, 0.9789, 0.2527, 0.3289, 0.6035, 0.7006, 0.5746, 0.4883]

评估LOSS: [4.869502067565918]

Plotting the loss curve

```
In [21]: plt.figure(figsize=(10, 5))
plt.plot(loss_callback.steps, loss_callback.train_losses, label="Train Loss")
if loss_callback.eval_losses:
    plt.plot(loss_callback.steps[:len(loss_callback.eval_losses)], loss_callback
plt.xlabel("Steps")
plt.ylabel("Loss")
plt.title("Loss Curve")
plt.legend()
plt.grid(True)
plt.savefig("loss.png")
plt.show()
```



Part 5: Evaluation and Results

Calculate the test set perplexity

```
In [31]: import math
perplexity = math.exp(loss_callback.eval_losses[0])
print(f"Perplexity: {perplexity:.2f}")
```

Perplexity: 130.26

Qualitative evaluation – Text Generation*

```
In [24]: del model
del tokenizer

import torch, gc
gc.collect()
torch.cuda.empty_cache()
torch.cuda.ipc_collect()

from peft import AutoPeftModelForCausalLM
from transformers import AutoTokenizer

NewModelDir = "./NewModel"

model = AutoPeftModelForCausalLM.from_pretrained(
    output_dir,
    device_map="auto",
    trust_remote_code=True
).eval()
tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)

merged_model = model.merge_and_unload()
merged_model.save_pretrained(NewModelDir, max_shard_size="2048MB", safe_serializati
tokenizer.save_pretrained(NewModelDir, max_shard_size="2048MB", safe_serializati
```



```

2025-04-21 12:08:23,809 - [WARNING] - The model is automatically converting to bf
16 for faster inference. If you want to disable the automatic precision, please m
anually add bf16/fp16/fp32=True to "AutoModelForCausalLM.from_pretrained".
2025-04-21 12:08:23,812 - [WARNING] - Try importing flash-attention for faster in
ference...
2025-04-21 12:08:23,825 - [WARNING] - Warning: import flash_attn rms_norm fail, p
lease install FlashAttention layer_norm to get higher efficiency https://github.c
om/Dao-AILab/flash-attention/tree/main/csrc/layer_norm
/home/ubuntu/anaconda3/envs/yx/lib/python3.9/site-packages/accelerate/utils/model
ing.py:1569: UserWarning: Current model requires 1572992 bytes of buffer for offl
oaded layers, which seems does not fit any GPU's remaining memory. If you are exp
erienceing a OOM later, please consider using offload_buffers=True.
  warnings.warn(
2025-04-21 12:08:24,343 - [INFO] - Based on the current allocation process, no mo
dules could be assigned to the following devices due to insufficient memory:
  - 0: 1244659712 bytes required
These minimum requirements are specific to this allocation attempt and may vary.
Consider increasing the available memory for these devices to at least the specif
ied minimum, or adjusting the model config.
Loading checkpoint shards: 100%|██████████| 2/2 [00:03<00:00, 1.57s/it]
2025-04-21 12:08:31,809 - [INFO] - Based on the current allocation process, no mo
dules could be assigned to the following devices due to insufficient memory:
  - 0: 1244659712 bytes required
These minimum requirements are specific to this allocation attempt and may vary.
Consider increasing the available memory for these devices to at least the specif
ied minimum, or adjusting the model config.

```

```

Out[24]: ('./NewModel/tokenizer_config.json',
          './NewModel/special_tokens_map.json',
          './NewModel/qwen.tiktoken',
          './NewModel/added_tokens.json')

```

```

In [28]: from modelscope import AutoModelForCausalLM, AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained(NewModelDir, trust_remote_code=True)

model = AutoModelForCausalLM.from_pretrained(NewModelDir, device_map="auto", tru

questions = [
    "有没有哪句诗能形容太阳初升时的壮观景象？",
    "你知道哪首诗描绘了金陵古城的险峻与历史厚重感吗？",
    "有没有一首诗能描绘雪花飘落的静谧和广袤天地的景象？",
    "有没有描写云的动态变化和神秘感的古诗推荐？",
    "有没有描写春天到来、万物复苏的古诗？"
]
answers_new = []
answers_old = []
for question in questions:
    response, _ = model.chat(tokenizer, question, history=None)
    answers_new.append(response)

del model
del tokenizer

import torch, gc
gc.collect()
torch.cuda.empty_cache()
torch.cuda.ipc_collect()

model = AutoModelForCausalLM.from_pretrained(model_name, device_map="auto", trus
tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)

```

```

for question in questions:
    response, _ = model.chat(tokenizer, question, history=None)
    answers_old.append(response)

for i in range(len(answers_new)):
    print(f"问题: {questions[i]}")
    print(f"新模型回答: {answers_new[i]}")
    print(f"旧模型回答: {answers_old[i]}")
    print("==" * 20)

```

```

2025-04-21 12:15:08,154 - [WARNING] - Warning: import flash_attn rms_norm fail, please install FlashAttention layer_norm to get higher efficiency https://github.com/Dao-AILab/flash-attention/tree/main/csrc/layer_norm
2025-04-21 12:15:08,487 - [INFO] - We will use 90% of the memory on device 0 for storing the model, and 10% for the buffer to avoid OOM. You can set `max_memory` in to a higher value to use more memory (at your own risk).
Loading checkpoint shards: 100%|██████████| 2/2 [02:39<00:00, 79.78s/it]
2025-04-21 12:18:00,077 - [WARNING] - The model is automatically converting to bf16 for faster inference. If you want to disable the automatic precision, please manually add bf16/fp16/fp32=True to "AutoModelForCausalLM.from_pretrained".
2025-04-21 12:18:00,078 - [WARNING] - Try importing flash-attention for faster inference...
2025-04-21 12:18:00,083 - [WARNING] - Warning: import flash_attn rms_norm fail, please install FlashAttention layer_norm to get higher efficiency https://github.com/Dao-AILab/flash-attention/tree/main/csrc/layer_norm
2025-04-21 12:18:00,400 - [INFO] - We will use 90% of the memory on device 0 for storing the model, and 10% for the buffer to avoid OOM. You can set `max_memory` in to a higher value to use more memory (at your own risk).
Loading checkpoint shards: 100%|██████████| 2/2 [00:10<00:00, 5.40s/it]

```

问题：有没有哪句诗能形容太阳初升时的壮观景象？

新模型回答：未离海底千山黑，才到天中万国明。—— 卞震《句 其二》

旧模型回答："日出东方红似火，暖阳照耀万物生。"

问题：你知道哪首诗描绘了金陵古城的险峻与历史厚重感吗？

新模型回答：北枕大江枕玉关，南来佳丽必神仙。几番战砌朱栏迥，二十四桥柳色鲜。—— 徐铉《金陵览古 连理桥》

旧模型回答：一首描绘金陵古城险峻与历史厚重感的古诗是唐代诗人杜甫的《登高》。这首诗通过描述秋天景色和感叹人生离别，传达出作者对祖国山河的热爱和对生命无常的感慨，从而展现了南京这座城市的险峻与历史厚重感。

问题：有没有一首诗能描绘雪花飘落的静谧和广袤天地的景象？

新模型回答：片片飞来静又闲，楼头江上复山前。繁阴掩日依杨柳，密雪连天入薜萝。暗逐彩云空自舞，静临浑沌亦何言。别时京都汴梁下，只有萧骚半不还。——徐铉《减字木兰花 其二》

旧模型回答：当然有，以下是一首关于雪花飘落的静谧和广袤天地的景象的诗：

《雪》

北风卷地白草折，胡天八月即飞雪。

千树万树梨花开，忽如一夜春风来。

问题：有没有描写云的动态变化和神秘感的古诗推荐？

新模型回答：触目迷离幻若浮，欲飞无力纷如流。须臾总要添华发，始见朝昏有晦无。—— 释延寿《积雨》

旧模型回答：当然有，以下是一些描绘云的动态变化和神秘感的古诗：

1. 李白《静夜思》：

床前明月光，疑是地上霜。

举头望明月，低头思故乡。

2. 王之涣《登鹳雀楼》：

白日依山尽，黄河入海流。欲穷千里目，更上一层楼。

3. 范仲淹《渔家傲·秋思》：

塞下秋来风景异，衡阳雁去无留意。四面边声连角起，千嶂里，长烟落日孤城闭。

4. 马致远《天净沙·秋思》：

枯藤老树昏鸦，小桥流水人家。夕阳西下，断肠人在天涯。

5. 辛弃疾《青玉案·元夕》：

东风夜放花千树，更吹落、星如雨。宝马雕车香满路，凤箫声动，玉壶光转，一夜鱼龙舞。

6. 刘禹锡《乌衣巷》：

朱雀桥边野草花，乌衣巷口夕阳斜。旧时王谢堂前燕，飞入寻常百姓家。

以上都是描绘云动态变化和神秘感的古诗词，希望对你有帮助。

问题：有没有描写春天到来、万物复苏的古诗？

新模型回答：仲春初景物欣然，惠远恩深键户临。杨柳径薰行税利，牡丹枝上发新妍。流金水石沈荒堑，积草烟芜入麦田。岁岁今朝好时机，日迟时早尽精研。—— 徐铉《奉和御制早春》

旧模型回答：当然有，以下是一首描写春天到来、万物复苏的古诗：

《清明》

清明时节雨纷纷，

路上行人欲断魂。

借问酒家何处有？

牧童遥指杏花村。

=====

Self analysis/comments

Advantages: It has strong instruction-following capabilities and can output in the format of the QA pair of the fine-tuning dataset.

Disadvantages: There is an illusion problem, and the output content is not knowledge that exists in the objective world.